MOMO Quant

Part 9 — Testing & Quality Assurance Plan

Version: 1.0 Draft
Primary Goal: Ensure MOMO Quant is testable, reliable, explainable, and safe before any live trading is considered.

---

1. Testing Principle

MOMO Quant must not be judged by whether the code compiles.

That is too weak.

The system must be judged by whether:

```
It calculates correctly.
It stores decisions correctly.
It rejects unsafe trades correctly.
It simulates execution realistically.
It explains decisions clearly.
It handles failure safely.
It prevents accidental live trading.
```

A trading system with weak tests is dangerous.

The most important areas to test are:

```
Risk engine
Execution simulation
Backtesting engine
Strategy logic
Indicator calculations
Mode safety
Emergency stop
AI failure handling
```

---

2. Testing Strategy

Use a layered testing approach:

```
Unit Tests
    ↓
Integration Tests
    ↓
API Tests
    ↓
UI Tests
    ↓
End-to-End Workflow Tests
    ↓
Manual Replay Validation
    ↓
Paper Trading Validation
```

Each layer catches different problems.

Do not rely only on manual testing.

---

3. Test Projects

Backend test projects:

```
MomoQuant.UnitTests
MomoQuant.IntegrationTests
```

Python AI tests:

```
momo-ai/app/tests/
```

Frontend tests can be added later:

```
momo-dashboard/src/tests/
```

MVP priority should be backend and AI tests first.

The frontend matters, but bad trading logic is more dangerous than a broken table filter.

---

4. Unit Testing

Unit tests should validate isolated logic.

Required unit test areas:

```
Indicator calculations
Strategy signal generation
Risk rule evaluation
Position sizing
PnL calculations
Order fill simulation
Backtest performance calculations
AI confidence scoring
Market regime detection
Timeframe helpers
Decimal precision helpers
```

---

5. Indicator Tests

Indicators must be deterministic.

Test:

```
EMA
VWAP
RSI
ATR
Volume SMA
Swing high detection
Swing low detection
Market structure detection
```

Example test cases:

```
EMA should return expected value for known candle series.
ATR should increase when candle ranges expand.
```

```
VWAP should match manual calculation.
RSI should stay within 0–100.
Swing high should be detected only when surrounding candles confirm it.
```

Bad indicator logic will poison every strategy.

---

6. Strategy Tests

Each strategy must be tested independently.

Strategies:

```
Liquidity Sweep
VWAP Mean Reversion
EMA Pullback
```

Tests must verify:

```
Correct signal is generated.
No signal is generated in invalid conditions.
Signal direction is correct.
Signal reason is clear.
Strategy respects supported regimes.
Strategy respects supported timeframes.
Strategy does not place orders.
Strategy does not approve risk.
```

Example:

```
EMA Pullback should generate Long signal when:
EMA20 > EMA50 > EMA200,
price pulls back near EMA20,
bullish confirmation candle closes,
volume is acceptable.
```

Example rejection:

```
EMA Pullback should return NoTrade when market regime is Choppy.
```

---

7. AI Tests

The AI service must be tested like any other service.

Required tests:

```
Market regime detection
Confidence scoring
Anomaly detection
Trade explanation
Invalid input handling
Missing candle handling
Extreme value handling
Schema validation
Timeout/fallback behavior
```

Example tests:

```
Trending candle set returns Trending.
Overlapping candle set returns Ranging or Choppy.
High spread returns anomaly.
Weak strategy alignment returns low confidence.
Strong trend alignment returns high confidence.
Invalid candle payload returns validation error.
```

AI must be explainable and predictable in MVP.

Do not treat AI output as magic.

---

8. Risk Engine Tests

Risk engine tests are mandatory.

This is the most important test category.

Test rules:

```
MaxRiskPerTradePercent
MaxDailyLossPercent
MaxWeeklyLossPercent
MaxOpenPositions
```

```
MaxExposurePerSymbol
MaxCorrelatedExposure
MaxConsecutiveLosses
MinConfidenceScore
MaxSpreadPercent
MaxVolatility
EmergencyStopEnabled
```

Required cases:

```
Trade is approved when all rules pass.
Trade is rejected when confidence is too low.
Trade is rejected when daily loss limit is hit.
Trade is rejected when max open positions is reached.
Trade is rejected when emergency stop is active.
Position size is adjusted when confidence/risk requires it.
Rejected rule key is stored.
Risk reason is human-readable.
```

If the risk engine fails, the system is unsafe.

---

9. Execution Simulation Tests

Maker-order execution is tricky.

Tests must cover:

```
Limit order created.
Post-only order submitted.
Order fills when price touches limit.
Order does not fill when price moves away.
Order expires after timeout.
Partial fill is handled.
Fees are calculated correctly.
Missed order is logged.
Cancelled order does not create trade.
Filled order creates trade/position update.
```

Maker-only scalping can look profitable in fake backtests if fills are simulated badly.

This is a major danger.

---

10. Backtesting Tests

Backtesting must be tested carefully.

Required tests:

```
Backtest loads candles in correct order.
Indicators are calculated before strategies run.
Strategies run once per candle.
AI decision is stored.
Risk decision is stored.
Orders are simulated.
Trades are created.
PnL is calculated.
Fees are deducted.
Missed orders are logged.
Backtest result metrics are correct.
```

Also test:

```
No candles available.
Duplicate candles.
Missing indicator snapshots.
AI service unavailable.
Strategy disabled.
Risk profile missing.
Invalid date range.
```

Backtests must fail clearly, not silently produce nonsense.

---

11. Replay Tests

Replay mode must reuse core logic.

Tests:

```
Replay session is created.
Replay starts from correct candle.
Replay steps forward one candle.
Replay pauses correctly.
Replay resumes correctly.
```

```
Replay speed changes correctly.
Replay emits timeline events.
Replay decision includes signal, AI decision, risk decision, and execution
decision.
No-trade reasons are shown.
```

Replay is the debugging microscope of the system.

If replay is weak, debugging will be painful.

---

12. Paper Trading Tests

Paper trading must prove that live-style operation works without real money.

Tests:

```
Paper account is created.
Paper trading starts.
Live market data is consumed.
Signals are generated.
AI is called.
Risk is evaluated.
Simulated order is created.
Position is updated.
Paper account balance changes.
Paper trading stops safely.
No real exchange order is placed.
```

Also test:

```
Exchange data stops.
AI service fails.
Redis unavailable.
Emergency stop triggered.
Daily loss limit hit.
```

Paper trading must behave close to live trading.

---

13. API Integration Tests

Integration tests should verify APIs with real database test container if possible.

Required API tests:

```
Auth login
Current user
User creation
Exchange creation
Symbol sync
Candle import
Indicator recalculation
Strategy enable/disable
Risk profile creation
Backtest run
Replay create/start
Paper account create
Reports load
Monitoring health
Audit logs
```

Use a test database.

Do not run integration tests against production or development data.

---

14. Database Tests

Database tests must verify:

```
Migrations apply.
Indexes exist.
Candle uniqueness works.
Decimal precision is preserved.
Relationships are valid.
Delete restrictions protect trading records.
Audit logs are inserted.
UTC timestamps are saved.
```

Trading data should not be casually deleted.

---

15. Security Tests

Security tests should verify:

```
Unauthenticated users cannot access protected APIs.
Viewer cannot start bot.
Trader cannot enable live trading.
Only Admin can manage users.
Only Admin can manage API keys.
JWT expiration is respected.
API keys are never returned raw.
Sensitive settings are not exposed.
Dangerous actions create audit logs.
```

For MVP, focus on role enforcement and secret handling.

---

16. Mode Safety Tests

Mode safety is critical.

Test:

```
Default mode is not Live.
Live mode cannot be enabled without readiness.
Live mode requires Admin.
Live mode requires confirmation text.
Live mode remains blocked if paper validation is missing.
Live mode remains blocked if risk profile is missing.
Live mode remains blocked if system health is bad.
Emergency stop blocks new trades.
Clearing emergency stop requires Admin.
```

The app must make accidental live trading difficult.

---

17. Reporting Tests

Reports must be based on stored facts.

Test:

```
Total PnL calculation
Net PnL calculation
Fee calculation
Win rate
Profit factor
Expectancy
Average win
Average loss
Max drawdown
Strategy performance
Symbol performance
Market regime performance
Rejected reason counts
Missed order counts
AI confidence bucket performance
```

Bad reporting leads to bad decisions.

---

18. Monitoring Tests

Monitoring tests should verify:

```
API health returns status.
Database health is checked.
Redis health is checked.
AI service health is checked.
Exchange health is checked.
Worker status is visible.
Stale market data is detected.
Critical health failure blocks trading if configured.
```

The system should not trade blindly when dependencies are broken.

---

19. Frontend Testing

Frontend tests can be lighter in MVP, but key flows must be tested.

Required UI checks:

```
Login page works.
Dashboard loads.
Mode badge is visible.
Emergency stop is visible.
Bot status is visible.
Backtest form validates inputs.
Replay controls are visible.
Paper mode is clearly labeled.
Live mode is locked.
Dangerous actions require confirmation.
Reports display data.
API errors show useful messages.
```

Manual testing is acceptable early, but core UI flows should eventually have automated tests.

---

20. End-to-End Local Test Flow

The first full local test should be:

```
1. Start MySQL and Redis.
2. Run backend migrations.
3. Start .NET API.
4. Start Python AI service.
5. Start React dashboard.
6. Login as Admin.
7. Create or seed exchange.
8. Sync or seed symbols.
9. Import BTCUSDT 3m candles.
10. Recalculate indicators.
11. Enable EMA Pullback strategy.
12. Create conservative risk profile.
13. Run backtest.
14. View signals.
15. View AI decisions.
16. View risk decisions.
17. View orders/trades.
18. View performance report.
19. Create replay session.
20. Step through replay decisions.
21. Create paper account.
22. Start paper trading.
```

```
23. Trigger emergency stop.
24. Confirm paper trading stops safely.
```

If this flow fails, the MVP is not ready.

---

21. Smoke Test Checklist

After every meaningful deployment or major change:

```
Backend builds.
Frontend builds.
AI service starts.
Database migration works.
Login works.
Dashboard loads.
Health endpoint is healthy.
Symbols load.
Strategies load.
Risk profiles load.
Backtest starts.
Replay starts.
Paper account loads.
SignalR connects.
Emergency stop works.
```

Smoke tests catch obvious failures fast.

---

22. Regression Testing

Whenever a module changes, retest affected flows.

Examples:

```
If indicator logic changes, rerun strategy and backtest tests.
If risk logic changes, rerun all risk and paper trading tests.
If execution simulation changes, rerun backtest performance tests.
If AI scoring changes, rerun confidence bucket reports.
If database schema changes, rerun migrations and reports.
```

Do not assume unrelated modules are unaffected.

Trading systems are connected.

---

23. Test Data Strategy

Create controlled test datasets.

Required datasets:

```
Trending market candles
Ranging market candles
Choppy market candles
High-volatility candles
Liquidity sweep example candles
VWAP reversion example candles
EMA pullback example candles
Missing candle dataset
Duplicate candle dataset
Extreme spread dataset
```

These datasets make tests repeatable.

Do not rely only on random live market data.

---

24. Backtest Validation Rules

Backtest results must be treated skeptically.

Before trusting a strategy:

```
Test multiple symbols.
Test multiple months.
Test different market regimes.
Include fees.
Include spread.
Include maker-fill assumptions.
Track missed orders.
Avoid optimizing on one period only.
Compare in-sample vs out-of-sample.
```

Backtests that ignore execution reality are fiction.

25. Paper Trading Validation Rules

Before live trading is even considered, paper trading should prove:

```
The bot can run continuously.
Market data stays connected.
Signals are reasonable.
Risk rules work.
Emergency stop works.
Reports match stored trades.
AI confidence has useful correlation with outcomes.
Maker order miss rate is understood.
No severe unexplained behavior occurs.
```

Recommended minimum before live:

```
At least several weeks of stable paper trading.
Multiple backtests across different periods.
Replay review of winning and losing sessions.
Manual review of risk rejections.
Manual review of missed orders.
```

Do not rush this.

---

26. Performance Testing

MVP performance tests should check:

```
Candle import speed.
Indicator recalculation speed.
Backtest runtime.
Report query speed.
Dashboard response time.
SignalR event volume.
AI service latency.
Database query performance.
```

Initial limits:

```
Top 5 symbols
3m and 5m timeframes
15m higher timeframe
Reasonable date ranges
```

Optimize only after measuring.

---

27. Failure Testing

The system must handle failure.

Test:

```
AI service down.
Redis down.
Database unavailable.
Exchange API down.
WebSocket stale.
Invalid candle data.
Order simulation failure.
Report generation failure.
JWT expired.
User lacks permission.
Emergency stop triggered during active paper trade.
```

Expected behavior:

```
Fail clearly.
Log error.
Protect trading.
Do not silently continue in unsafe state.
```

---

28. CI Testing

Use GitHub Actions later.

Initial CI checks:

```
Backend build
Backend unit tests
AI service tests
Frontend build
Docker build
```

Do not overbuild CI at the start.

But at minimum, code should not merge if it does not build.

---

29. Manual QA Checklist

Before marking any milestone complete:

```
Does the module compile?
Does the API work?
Does the UI show the correct data?
Are errors handled?
Are audit logs created?
Are timestamps UTC?
Are decimals used for financial values?
Are dangerous actions protected?
Are tests added for important logic?
Is documentation updated?
```

A milestone is not done just because Cursor generated files.

---

30. Definition of Done

A feature is done only when:

```
It builds.
It passes tests.
It handles bad input.
It logs important actions.
It uses standard API response format.
It respects permissions.
It does not violate architecture rules.
It is visible in the dashboard if needed.
```

```
It has clear failure behavior.
It can be manually verified.
```

For trading logic, also require:

```
It stores decisions.
It explains decisions.
It respects risk engine authority.
It does not bypass safety controls.
```

---

31. Testing Priority

Highest priority:

```
Risk Engine
Execution Simulation
Backtesting Engine
Mode Safety
Emergency Stop
Indicator Calculations
Strategy Signals
AI Failure Handling
```

Medium priority:

```
Reports
Replay
Paper Trading
Monitoring
Audit Logs
API Integration
```

Lower priority for MVP:

```
Advanced UI tests
Visual regression tests
Load testing
Complex CI/CD pipelines
```

---

32. Final QA Summary

Testing in MOMO Quant is not paperwork.

It is the difference between a research platform and a dangerous script.

The most important testing rule:

```
Never trust a trade result unless you can trace the full path:
Candle → Indicator → Strategy Signal → AI Decision → Risk Decision → Order →
Fill → Trade → Report
```

If any link in that chain is missing, the system is not ready.